Flaris FFI - Foreign Function Interface

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.6.0 **Generated** 2026-10-24 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Table of Contents

1. Overview
2. Security Model
3. The Ffi Module
4. Type Mapping
5. FfiObject Reference
6. Writing a Plugin
7. Ownership & Lifetime Rules
8. Helper Macros
9. Complete Plugin Template
10. Step-by-Step Tutorial
11. Advanced Examples
12. Performance Guidelines
13. Callbacks - Calling Back into Flaris
14. Returning Functions from Flaris FFI
15. Best Practices & Safety

---

1. Overview

The `Ffi` module lets Flaris scripts call functions inside native shared libraries (`.so`, `.dylib`, `.dll`). Arguments and return values are automatically marshalled through a structured type system - no raw pointer casting required on the Flaris side.

What FFI is for:

- Performance-critical native routines (SIMD, compression, hashing)
- Platform or hardware-specific APIs
- Wrapping an existing C library you don't want to rewrite
- Large binary data processing

What FFI is not for:

- Per-element inner loops (marshalling overhead is non-trivial)

Flaris does not call arbitrary C functions. Every function it can call has the one signature `FfiObject f(FfiObject *args, int argc)`, so an existing library is reached through a small shim you compile against

`ffi_object.h` - one exported wrapper per entry point you want (`ffi/sqlite_ffi.c` is a worked example). Calling `libcurl` or `libc` directly, without that shim, is not supported.

---

2. Security Model

FFI is **disabled by default**. It must be explicitly enabled, with one of two flags:

```
flarisvm --allow-ffi script.fls    # Ffi.* only
flarisvm --unsafe    script.fls    # Ffi.* and everything else below
```

Without either, every `Ffi.*` call raises `Exception.UnsafeOperation` at runtime. The same holds for calling a function `Ffi.GetFunction` returned: it runs only where FFI is allowed, so handing one to an embedded context or fiber without it raises `UnsafeOperation` there (`ModuleDenied` when the host withholds `Ffi`).

Prefer `--allow-ffi`. `--unsafe` is a single switch over four separate capabilities, so granting FFI with it also grants the other three:

capability	granted by `--allow-ffi`	granted by `--unsafe`
`Ffi.*` - load and call native code	yes	yes
`Memory.*`, `Buffer.GetAddress` - read/write any address	no	yes
`import` over plain `http://`	no	yes
raw JIT entry points	no	yes

A script that needs a database plugin needs the first row and none of the rest. `--unsafe` keeps its existing meaning and still implies `--allow-ffi`, so existing command lines, `.flx` embed headers and `cfg.unsafe` are unaffected.

Neither flag makes a plugin safe: native code runs in your process with your privileges, and `--allow-ffi` only narrows what the *script* around it may do.

Library fingerprinting. When signature checks are on (default), `Ffi.Load` computes the SHA-256 of the library file on every load and compares it to the hash you provide. A mismatch raises `Exception.ChecksumError` and the library is not loaded. This prevents tampered or swapped libraries from being loaded silently.

To get the hash of your compiled plugin:

```
flarisvm --sha256 plugin.so          # built-in fingerprint tool
# or
sha256sum plugin.so            # standard system tool (same algorithm)
```

Signature checks can be disabled with `--no-verify`, but this is not recommended outside controlled environments.

---

3. The Ffi Module

Namespace: **`Ffi`** - requires `--allow-ffi` (or `--unsafe`, which implies it).

Function	Signature	Description
Load	`Load(path:string, sha256:string|nil, flags?:int) – pointer`	Load shared library at `path`. When `sha256` is a 64-character hex string the library's fingerprint is verified before loading; pass `nil` to skip the check. A digest that is not 64 hex characters is treated as a malformed pin and raises `Exception.ChecksumError` rather than loading unverified - as does a fingerprint mismatch. `flags` may combine `Ffi.LAZY` (default), `Ffi.NOW` and `Ffi.GLOBAL`; ignored on Windows. Returns a library handle (`pointer`), or `nil` if the file cannot be loaded - call `Ffi.LastError()` for the reason. Libraries are cached - loading the same path twice returns the same handle, and stay loaded for the lifetime of the process.
GetFunction	`GetFunction(handle:pointer, name:string, signature?:string) – ffi_function`	Resolve a named symbol from a loaded library. Returns a callable FFI function, or `nil` if the symbol is not found or the signature is malformed. The optional `signature` takes the form `fn(arg1,arg2,...):returnType`, for example `fn(int,int):int` or `fn():nil`; argument count, argument types **and the return type** are then validated on every call. A return that does not match raises `Exception.TypeMismatch`; `nil` is always accepted, so a plugin can signal failure with `nil` whatever type it declares. Omit the signature to leave the call fully dynamic.
HasFunction	`HasFunction(handle:pointer, name:string) – bool`	Returns `true` if the named symbol exists in the library. Does not allocate an object. Use to guard optional symbols before calling `GetFunction`.
SetCallback	`SetCallback(handle:pointer, name:string, fn:function) – bool`	Register a Flaris function as a named callback for the library. The library must export `flaris_set_callback`. Registrations are scoped to `handle`, so two libraries may use the same name without colliding; a name is resolved against the library whose code is calling back. Re-registering the same name for the same library replaces the previous function.
RemoveCallback	`RemoveCallback(handle:pointer, name:string) – bool`	Unregister a single named callback without unloading the library. Useful for one-shot callbacks.
GetVersion	`GetVersion(handle:pointer) – int`	Returns the plugin version as `uint32` (`0xMMmmppbb`). Plugin must export `uint32_t flaris_version(void)`. Returns `0` if symbol not found.
Call	`Call(handle:pointer, name:string, ...args) – any`	Resolve and call a named symbol in one step. Up to 4 extra arguments. Calls `dlsym` on every invocation - use `GetFunction` for hot paths.
CallAsync	`CallAsync(fn:ffi_function, args:array) – any`	Run a **pure** FFI function on the I/O thread pool so the calling fiber parks instead of freezing the scheduler. Must be `await`ed; `args` is the argument array; block arguments are refused. See *Async calls* below.
LastError	`LastError() – string|nil`	Text of the most recent `Load` / `GetFunction` / `Call` failure, or `nil` if the last one succeeded. Those three clear it on entry, so it always describes the most recent call - it does not linger errno-style.
LAZY / NOW / GLOBAL	`int`	`dlopen` flag constants for `Load`. `LAZY` (default) resolves symbols on first use; `NOW` resolves everything at load, turning a missing symbol into a failed `Load` instead of a crash on first call; `GLOBAL` exposes the library's symbols to libraries loaded afterwards.

All libraries still open at VM shutdown are automatically closed.

Library resolution

The first argument to `Ffi.Load` is resolved in this order:

1. **As given** - an absolute or relative path that exists is loaded directly (`Ffi.Load("./build/plugin.so", …)`).
2. **Per-user FFI directory** - `~/.flaris/ffi` (POSIX) / `%USERPROFILE%\.flaris\ffi` (Windows), overridable with the `FLARIS_FFI` environment variable. A **bare name** (no path separator, no extension) is looked up here with the platform extension appended automatically, so `Ffi.Load("myplugin")` finds `~/.flaris/ffi/myplugin.dylib` / `.so` / `.dll`. A given path that does not exist also falls back to `<ffi-dir>/<basename>`.
3. **Working directory** - `<name>` with the platform extension appended.
4. **System loader** - if nothing above matches, the name is handed to the OS loader, so a system SONAME such as `"libsqlite3.so"` still works.

The portable, recommended form is therefore a **bare name**: install the C library once into `~/.flaris/ffi` (a package manager does this for you) and load it with `Ffi.Load("myplugin")` on every platform - no `./libs/...` path and no per-OS extension handling. The bundled FFI libraries (SQLite, Redis, Postgres) use this form.

Function Signature Format

Signatures passed to `GetFunction` use the syntax:

```
fn(arg_type, arg_type, ...):return_type
```

Types may be unioned with `|`. Zero-argument functions use `fn():return_type`.

Type name	Matches
`any`	Any value
`nil`	Nil
`bool`	Boolean
`int`	Integer (`u8` , `u16` , `u32` , `u64` , `i8` , `i16` , `i32` , `i64` are aliases)
`float`	Float
`char`	Character
`string`	String
`array`	Array
`object`	Object
`block`	Block (raw memory buffer)
`pointer` / `ptr`	Pointer
`function` / `fn`	Flaris function
`builtin`	Built-in function
`ffi`	FFI function
`number` / `numeric`	`int|float`
`callable`	`fn|builtin|ffi`

Examples:

```
let fn = Ffi.GetFunction(lib, "add", "fn(int, int):int");
let fn = Ffi.GetFunction(lib, "process", "fn(string, pointer):bool");
let fn = Ffi.GetFunction(lib, "compute", "fn(number):float");
let fn = Ffi.GetFunction(lib, "reset", "fn():nil");
let fn = Ffi.GetFunction(lib, "anything", "fn(any):any");
```

Basic usage pattern

```
// Load
let lib = Ffi.Load("./plugin.so", "a3f1...64hexchars...");
if (lib == nil) {
    Console.Error("Failed to load plugin");
    VM.Exit(1);
}

// Resolve functions
let Add = Ffi.GetFunction(lib, "ffi_add");
let Greet = Ffi.GetFunction(lib, "ffi_greet", "fn(string):nil");

// Call - identical to a regular Flaris function call
let result = Add(10, 32);        // - 42
let msg = Greet("Flaris");       // - "Hello, Flaris!"

// No unload step: a loaded library stays mapped for the process lifetime
// and is closed by the VM at exit.
```

Version checking

Plugins can export a version number using the same `0xMMmmppbb` format. Flaris can verify compatibility before calling any other functions:

```
let ver = Ffi.GetVersion(lib);
if (ver == 0) {
    Console.WriteLine("plugin does not export version");
} else if (ver < 0x01000002) {
    Console.Error("plugin too old, need at least 1.0.0.2");
    VM.Exit(1);
}
```

Plugin side:

```
uint32_t flaris_version(void) {
    return 0x01000001;  // 1.0.0.1
}
```

One-off calls with Ffi.Call

For infrequent calls, `Ffi.Call` resolves and invokes a symbol in one step without allocating an `ffi_function` object:

```
// Single call — clean and allocation-free
let result = Ffi.Call(lib, "ffi_add", 10, 32);
```

`Ffi.Call` calls `dlsym` on every invocation. For functions called repeatedly, use `GetFunction` to cache the symbol:

```
// Hot path — resolve once, call many times
let add = Ffi.GetFunction(lib, "ffi_add");
foreach (x in data) { add(x); }
```

`Ffi.Call` accepts up to 4 arguments beyond the handle and name.

---

4. Type Mapping

When you call an FFI function, Flaris converts each argument from a VM value to an `FfiObject`. The return value is converted back. This table shows the mapping:

Flaris type	FfiObject type constant	Storage field	Notes
`nil`	`FFIVAL_NIL`	-	Nil sentinel
`bool`	`FFIVAL_BOOL`	`ival` (0 or 1)	
`char`	`FFIVAL_CHAR`	`ival` (codepoint)	Unicode scalar value
`int`	`FFIVAL_INT`	`ival` (int64_t)	Full 64-bit signed
`float`	`FFIVAL_FLOAT`	`fval` (double)	IEEE-754 64-bit
`string`	`FFIVAL_STRING`	`string.data` / `string.length`	Read-only view, valid only during the call - copy to keep, must not free or mutate
`array`	`FFIVAL_ARRAY`	`array.elements` / `array.length`	Deep-copied - all elements recursively converted
`object`	`FFIVAL_OBJECT`	`object.pairs` / `object.length`	Deep-copied key-value pairs
`block`	`FFIVAL_BLOCK`	`block.memory` / `block.block_size` / `block.block_count`	**Pointer into VM memory** - do not free
-	`FFIVAL_TABLE`	`table.col_names` / `table.cells` / `table.ncols` / `table.nrows`	Return-only rowset; arrives in script code as an array of objects
`pointer`	`FFIVAL_POINTER`	`pointer.addr` / `pointer.tag`	Opaque handle owned by the plugin; the VM copies it and never dereferences or frees it

Unsupported types (`fiber`, `function`, `class`, `instance`, `stream`) are passed as `FFIVAL_NIL`. If your function receives an unexpected nil, check that you passed a supported type.

Opaque handles

A plugin that owns a resource - a database connection, a decoder context, a socket - returns it as `FFIVAL_POINTER`. Script can hold the handle and pass it back, but cannot dereference it, do arithmetic on it, or

create one: there is no pointer literal and no conversion into `pointer`. Declaring the parameter as `ptr` therefore turns what used to be a segfault into a catchable exception:

```
let open  = Ffi.GetFunction(lib, "db_open",  "fn(string):ptr");
let query = Ffi.GetFunction(lib, "db_query", "fn(ptr, string):array");
let close = Ffi.GetFunction(lib, "db_close", "fn(ptr):bool");

let db = open("app.db");
query(db, "select 1");
query(12345, "select 1");   // raises — an int is not a pointer
close(db);
```

Before this existed the only way to return a handle was to cast it to `int`, which made every handle forgeable from any number.

`tag` is a plugin-chosen type id carried with the address, so a plugin handed the wrong *kind* of handle can reject it rather than casting blind. The VM only copies the tag; the plugin decides what it means.

```
#define TAG_DB 0x44425F31u   /* any nonzero constant, one per handle type */

FfiObject db_open(FfiObject *args, int argc) {
    sqlite3 *db = NULL;
    if (sqlite3_open(AS_STRING(args[0]), &db) != SQLITE_OK) return ffi_nil();
    return ffi_pointer(db, TAG_DB);
}

FfiObject db_close(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    if (!FFI_PTR_IS(args[0], TAG_DB)) return ffi_bool(0);  /* wrong handle type */
    sqlite3_close((sqlite3 *)AS_POINTER(args[0]));
    return ffi_bool(1);
}
```

The plugin owns the lifetime. The VM frees nothing when the handle goes out of scope, so a plugin that hands out handles must expose a close entry point, and script must call it. A handle used after close is a use-after-free the VM cannot detect - keep the tag check, but note it cannot catch this case.

Strings are passed zero-copy: `string.data` points at the live Flaris string (marked with the `FFI_FLAG_BORROWED` flag bit). It is valid for the duration of the call only - `memcpy` the bytes if you need them after your function returns, and never `free()` or write through the pointer. The container structure of **arrays and objects** (element and pair nodes) is still built per call and freed by the VM after the function returns, so large nested structures have proportional marshalling cost.

Blocks also alias VM memory directly (`block.memory`) - zero-copy and writable by convention, but the plugin must **never free or reallocate** that pointer. Total byte size is `block.block_size * block.block_count`. Blocks are by far the cheapest way to move bulk data across the boundary; prefer them over big arrays in hot paths.

---

5. FfiObject Reference

The `FfiObject` struct is defined in `ffi_object.h`. This is the only header your plugin needs to include.

```
typedef enum {
    FFIVAL_NIL,
    FFIVAL_CHAR,
    FFIVAL_FLOAT,
    FFIVAL_INT,
    FFIVAL_STRING,
    FFIVAL_OBJECT,
    FFIVAL_ARRAY,
    FFIVAL_BOOL,
    FFIVAL_BLOCK,
    FFIVAL_FUNC,
    FFIVAL_TABLE,
} FfiObjectType;

struct FfiObject {
    FfiObjectType type;
    uint32_t flags;       // FFI_FLAG_BORROWED on some args; 0 in values you build

    union {
        double   fval;    // FFIVAL_FLOAT
        int64_t  ival;    // FFIVAL_INT, FFIVAL_BOOL, FFIVAL_CHAR

        struct {
            char    *data;
            uint32_t length;
        } string;          // FFIVAL_STRING

        struct {
            FfiKV   *pairs;        // contiguous; each pair embeds its value
            uint32_t length;
        } object;          // FFIVAL_OBJECT

        struct {
            FfiObject *elements;   // contiguous value array, NOT pointers
            uint32_t   length;
        } array;           // FFIVAL_ARRAY

        struct {
            void    *memory;
            uint8_t  block_size;   // bytes per element
            uint32_t block_count;  // number of elements
        } block;           // FFIVAL_BLOCK
```

```
        struct {
            char     **col_names;  // ncols malloc'd names, owned by this node
            FfiObject *cells;      // nrows*ncols cells, row-major, owned
            uint32_t   ncols;
            uint32_t   nrows;
        } table;           // FFIVAL_TABLE
    };
};
```

`FfiKV` (object key-value pair):

```
struct FfiKV {
    char     *key;    // null-terminated, malloc'd string key
    FfiObject value;  // embedded by value, not a pointer
};
```

Arrays and object pairs hold their children **by value in one contiguous allocation**: build them with `ffi_array_n(n)` / `ffi_object_n(n)` and index directly (`arr.array.elements[i] = ffi_int(42);`). This keeps a container of any size at one allocation for the container plus whatever its string cells need.

Tables - the fast path for row-shaped data

`FFIVAL_TABLE` is a rowset: the column names are stated once and the cells are one contiguous row-major block (`cells[r * ncols + c]`). The runtime converts a table to an array of objects - script code receives `[{ col: value, ... }, ...]` - but each column name is transferred and interned once per result instead of once per row. Use it for anything row-shaped: database results, CSV data, spreadsheets.

```
FfiObject t = ffi_table_n(nrows, ncols);      // all cells nil, names NULL
for (uint32_t c = 0; c < ncols; c++)
    t.table.col_names[c] = strdup(names[c]);  // malloc'd, VM takes ownership
for (uint32_t r = 0; r < nrows; r++)
    for (uint32_t c = 0; c < ncols; c++)
        FFI_TBL_CELL(t, r, c) = ffi_int(...); // or ffi_string / ffi_float / ...
return t;
```

See `ffi/pg_ffi.c` and `ffi/sqlite_ffi.c` for complete examples.

Constructing return values

All fields not set should be zeroed. The cleanest approach is to zero-initialize and then fill in only what you need:

```
// Return an int
FfiObject out = {0};
out.type = FFIVAL_INT;
out.ival = 42;
return out;
```

```
// Return a float
FfiObject out = {0};
out.type = FFIVAL_FLOAT;
out.fval = 3.14;
return out;

// Return a bool
FfiObject out = {0};
out.type = FFIVAL_BOOL;
out.ival = 1;   // 1 = true, 0 = false
return out;

// Return nil (error / unsupported)
FfiObject out = {0};
out.type = FFIVAL_NIL;
return out;

// Return a string — you must malloc the buffer; the VM takes ownership
FfiObject out = {0};
out.type = FFIVAL_STRING;
out.string.data = strdup("result text");
out.string.length = strlen("result text");
return out;
```

String memory rule: Strings you return must be heap-allocated with `malloc` (or `strdup`, or the `ffi_string` / `ffi_string_n` helpers). Ownership transfers to the VM on return: it either adopts the buffer as the string's storage or frees it. Never return a pointer to a stack buffer, a string literal, or argument memory - and never touch the buffer again after returning it. Set `string.length` accurately; the runtime uses it.

---

6. Writing a Plugin

Required function signature

Every exported function must match this signature exactly:

```
FfiObject fn_name(FfiObject *args, int argc);
```

- `args` - array of `FfiObject` values, one per argument passed from Flaris
- `argc` - number of arguments actually passed
- return - a single `FfiObject` (by value)

Minimal example

```
// plugin.c
#include <string.h>
```

```
#include "ffi_object.h"

FfiObject ffi_add(FfiObject *args, int argc) {
    if (argc != 2 ||
        args[0].type != FFIVAL_INT ||
        args[1].type != FFIVAL_INT) {
        FfiObject nil = {0};
        nil.type = FFIVAL_NIL;
        return nil;
    }

    FfiObject out = {0};
    out.type = FFIVAL_INT;
    out.ival = args[0].ival + args[1].ival;
    return out;
}
```

Compiling

```
# Linux / macOS
gcc -shared -fPIC -o plugin.so plugin.c

# With debug symbols (development)
gcc -shared -fPIC -g -o plugin.so plugin.c

# Cross-compile for ARM (embedded)
arm-linux-gnueabihf-gcc -shared -fPIC -o plugin.so plugin.c
```

The plugin does **not** need to link against FlarisVM or any VM library. Only `ffi_object.h` is needed at compile time.

Installing. Copy the built library into the per-user FFI directory so it can be loaded by bare name on any platform (see *Library resolution* above):

```
mkdir -p ~/.flaris/ffi
cp plugin.so ~/.flaris/ffi/        # then: Ffi.Load("plugin")
```

Getting the SHA-256 fingerprint

After compiling, generate the hash for use in your Flaris script:

```
flarisvm --sha256 plugin.so
# Output: a3f19c...64 hex chars...
```

---

7. Ownership & Lifetime Rules

Understanding memory ownership is critical to writing correct plugins.

VM owns

- All values passed as `args` - do not free them
- `block.memory` inside `FFIVAL_BLOCK` args - do not free or reallocate
- All string `.data` fields inside args - read-only views of live VM strings (`FFI_FLAG_BORROWED`), valid only until your function returns; copy the bytes to keep them

Plugin owns (and is responsible for freeing)

- Any `malloc`/`strdup` allocations made inside the plugin
- The `string.data` of any `FFIVAL_STRING` you **return** - until the moment you return it; ownership then transfers to the VM (which adopts or frees it)

VM automatically releases

- The entire `FfiObject` argument tree after the call completes
- The returned `FfiObject` contents (e.g., `string.data`) after conversion back to a VM value
- All array element and object pair nodes it built for the call

Plugin must never

- Store `FfiObject *` pointers or argument `string.data` pointers past the function return
- Free any pointer from an argument
- Free or reallocate `block.memory`
- Modify `string.data` of an incoming argument
- Touch a returned buffer after returning it
- Access args after returning
- Attempt callbacks into the VM (outside the documented `flaris_set_callback` mechanism)

---

8. Helper Macros

`ffi_object.h` is self-contained: besides the type-check and accessor macros it provides `static inline` constructors for return values and a couple of container helpers, so most plugins need no hand-written boilerplate at all.

```
// Type checks (take FfiObject by value)
IS_NIL(v)      // v.type == FFIVAL_NIL
IS_CHAR(v)     // v.type == FFIVAL_CHAR
IS_INT(v)      // v.type == FFIVAL_INT
IS_FLOAT(v)    // v.type == FFIVAL_FLOAT
IS_STRING(v)   // v.type == FFIVAL_STRING
IS_BOOL(v)     // v.type == FFIVAL_BOOL
IS_ARRAY(v)    // v.type == FFIVAL_ARRAY
IS_OBJECT(v)   // v.type == FFIVAL_OBJECT
```

```
IS_BLOCK(v)    // v.type == FFIVAL_BLOCK
IS_FUNC(v)     // v.type == FFIVAL_FUNC
IS_NUMBER(v)   // int or float

// Value accessors (take FfiObject by value)
AS_INT(v)         // v.ival
AS_FLOAT(v)       // v.fval
AS_CHAR(v)        // v.ival (codepoint)
AS_STRING(v)      // v.string.data
AS_STRING_LEN(v)  // v.string.length
AS_BOOL(v)        // v.ival != 0
AS_NUMBER(v)      // double, regardless of int/float storage

// Container accessors
FFI_ARR_LEN(v)      // v.array.length
FFI_ARR_AT(v, i)    // v.array.elements[i]     (FfiObject value)
FFI_OBJ_LEN(v)      // v.object.length
FFI_OBJ_KEY(v, i)   // v.object.pairs[i].key
FFI_OBJ_VAL(v, i)   // v.object.pairs[i].value (FfiObject value)
FFI_BLOCK_BYTES(v)  // block_size * block_count (uint64_t)
FFI_BLOCK_PTR(v)    // v.block.memory
FFI_TBL_ROWS(v)     // v.table.nrows
FFI_TBL_COLS(v)     // v.table.ncols
FFI_TBL_NAME(v, c)  // v.table.col_names[c]
FFI_TBL_CELL(v, r, c) // cell at row r, column c (FfiObject lvalue)

// Return-value constructors (no boilerplate needed)
ffi_nil()                 // -> nil
ffi_int(int64_t)          // -> int
ffi_float(double)         // -> float
ffi_bool(int)             // -> bool (0/1)
ffi_char(int64_t)         // -> char (codepoint)
ffi_func(FFIFunc)         // -> callable ffi-function
ffi_string(const char*)   // -> string, malloc'd copy, VM takes ownership (NULL -> nil)
ffi_string_n(s, len)      // -> string of exactly len bytes
ffi_array_n(n)            // -> array of n nil elements, one allocation
ffi_object_n(n)           // -> object of n pairs (keys NULL, values nil)
ffi_table_n(nrows, ncols) // -> rowset, all cells nil, names NULL
ffi_object_get(o, "key")  // -> FfiObject* field of an object arg, or NULL

// Version helper: pack into the 0xMMmmppbb format flaris_version() returns
FLARIS_VERSION(maj, min, pat, bld)

// Guard macros (return ffi_nil() if condition fails)
REQUIRE_ARGC(n)      // if argc < n
REQUIRE_TYPE(i, t)   // if args[i].type != t
REQUIRE_NUMBER(i)    // if args[i] is not int or float
```

Notes:

- The guard macros expand to `return ffi_nil();` and assume the parameters are named `args` and `argc` (the required signature) - use them only inside an FFI function body.
- `ffi_string` / `ffi_string_n` and `ffi_object_get` use `malloc` / `strdup`. To build without libc (freestanding targets), define `FFI_OBJECT_NO_STDLIB` before including the header; the scalar constructors and all macros remain available.

Example using guards and constructors:

```
FfiObject ffi_multiply(FfiObject *args, int argc) {
    REQUIRE_ARGC(2);
    REQUIRE_TYPE(0, FFIVAL_INT);
    REQUIRE_TYPE(1, FFIVAL_INT);
    return ffi_int(args[0].ival * args[1].ival);
}

// Accepts int or float, returns a float:
FfiObject ffi_scale(FfiObject *args, int argc) {
    REQUIRE_ARGC(2);
    REQUIRE_NUMBER(0);
    REQUIRE_NUMBER(1);
    return ffi_float(AS_NUMBER(args[0]) * AS_NUMBER(args[1]));
}

uint32_t flaris_version(void) { return FLARIS_VERSION(1, 0, 0, 1); }
```

---

9. Complete Plugin Template

Save as `plugin.c`. Copy, extend, and trim as needed.

```
// plugin.c - Flaris FFI plugin template
//
// Build:
//   gcc -shared -fPIC -o plugin.so plugin.c
//
// Usage from Flaris:
//   let lib = Ffi.Load("./plugin.so", "<sha256>");
//   let Fn  = Ffi.GetFunction(lib, "ffi_add_ints");

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include "ffi_object.h"

// Constructors (ffi_int / ffi_float / ffi_string / ffi_nil / ...) and the
```

```
// REQUIRE_* guards come from ffi_object.h — no local helpers needed.

// ── Exports ──────────────────────────────────────────────────────────────

// ffi_add_ints(a:int, b:int) — int
FfiObject ffi_add_ints(FfiObject *args, int argc) {
    REQUIRE_ARGC(2);
    REQUIRE_TYPE(0, FFIVAL_INT);
    REQUIRE_TYPE(1, FFIVAL_INT);
    return ffi_int(args[0].ival + args[1].ival);
}

// ffi_sum_array(nums:array) — float
// Accepts an array of int or float elements.
FfiObject ffi_sum_array(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_ARRAY);

    double sum = 0.0;
    for (uint32_t i = 0; i < FFI_ARR_LEN(args[0]); i++) {
        FfiObject el = FFI_ARR_AT(args[0], i);
        if (IS_NUMBER(el)) sum += AS_NUMBER(el);
    }

    return ffi_float(sum);
}

// ffi_block_xor(buf:block) — int
// XOR of all bytes in a raw memory block.
FfiObject ffi_block_xor(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_BLOCK);

    uint8_t *bytes = (uint8_t *)FFI_BLOCK_PTR(args[0]);
    if (!bytes) return ffi_int(0);

    uint64_t total = FFI_BLOCK_BYTES(args[0]);
    uint8_t acc = 0;
    for (uint64_t i = 0; i < total; i++) acc ^= bytes[i];

    return ffi_int((int64_t)acc);
}

// ffi_greet(name:string) — string
FfiObject ffi_greet(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_STRING);
```

```
    const char *name = args[0].string.data ? args[0].string.data : "";

    size_t nlen = strlen(name);
    char *buf = (char *)malloc(nlen + 9); // "Hello, " + name + "!" + '\0'
    if (!buf) return ffi_nil();
    memcpy(buf, "Hello, ", 7);
    memcpy(buf + 7, name, nlen);
    memcpy(buf + 7 + nlen, "!", 2); // includes '\0'

    FfiObject out = ffi_string(buf);
    free(buf);                      // ffi_string made its own copy
    return out;
}
```

---

10. Step-by-Step Tutorial

Step 1 - Write `plugin.c`

Use the minimal example from section 6 or the template above. Start with just one function.

Step 2 - Compile

```
gcc -shared -fPIC -o plugin.so plugin.c
```

Step 3 - Get the fingerprint

```
flarisvm --sha256 plugin.so
```

Copy the 64-character hex string. You'll paste it into your Flaris script.

Step 4 - Write `demo.fls`

```
let lib = Ffi.Load("./plugin.so", "a3f19c...your-hash-here...");
if (lib == nil) {
    Console.Error("Failed to load plugin.so");
    VM.Exit(1);
}

let Add = Ffi.GetFunction(lib, "ffi_add_ints", "fn(int,int):int");
if (Add == nil) {
    Console.Error("Symbol ffi_add_ints not found");
    VM.Exit(1);
}
```

```
let r = Add(2, 40);
Console.WriteLine("2 + 40 =", r);   // 2 + 40 = 42
```

Step 5 - Run

```
flarisvm --unsafe demo.fls
```

Output:

```
2 + 40 = 42
```

Step 6 - Iterate

Add more exported functions. Recompile. Re-get the fingerprint (it changes every time the binary changes). Update your script.

---

11. Advanced Examples

Returning a string

```
FfiObject ffi_upper(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_STRING);

    const char *src = args[0].string.data;
    uint32_t    len = args[0].string.length;

    char *buf = (char *)malloc(len + 1);
    if (!buf) return ffi_nil();

    for (uint32_t i = 0; i < len; i++)
        buf[i] = (char)toupper((unsigned char)src[i]);
    buf[len] = '\0';

    FfiObject out = {0};
    out.type = FFIVAL_STRING;
    out.string.data   = buf;
    out.string.length = len;
    return out;
}
```

```
let Upper = Ffi.GetFunction(lib, "ffi_upper", "fn(string):string");
Console.WriteLine(Upper("hello"));   // HELLO
```

Reading an object

```
FfiObject ffi_rect_area(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_OBJECT);

    int64_t w = 0, h = 0;
    uint32_t n = args[0].object.length;

    for (uint32_t i = 0; i < n; i++) {
        const char *key = args[0].object.pairs[i].key;
        FfiObject  *val = &args[0].object.pairs[i].value;
        if (!key) continue;

        if (strcmp(key, "w") == 0 && val->type == FFIVAL_INT) w = val->ival;
        if (strcmp(key, "h") == 0 && val->type == FFIVAL_INT) h = val->ival;
    }

    return ffi_int(w * h);
}
```

```
let Area = Ffi.GetFunction(lib, "ffi_rect_area");
Console.WriteLine(Area({ w: 4, h: 6 }));   // 24
```

Processing raw binary data

```
FfiObject ffi_checksum(FfiObject *args, int argc) {
    REQUIRE_ARGC(1);
    REQUIRE_TYPE(0, FFIVAL_BLOCK);

    uint8_t  *mem   = (uint8_t *)args[0].block.memory;
    uint64_t  total = (uint64_t)args[0].block.block_size *
                      (uint64_t)args[0].block.block_count;

    uint32_t h = 0;
    for (uint64_t i = 0; i < total; i++)
        h = h * 31 + mem[i];

    return ffi_int((int64_t)h);
}
```

```
let buf = Buffer.Create(1024, 1);
Buffer.Fill(buf, 0xAB);
```

```
let Checksum = Ffi.GetFunction(lib, "ffi_checksum", "fn(block):int");
Console.WriteLine(Checksum(buf));
```

---

12. Performance Guidelines

FFI calls are not free. Each call pays:

- Argument marshalling (stack walk + type conversion, recursive for arrays/objects)
- `dlsym` was already resolved at `GetFunction` - no lookup per call
- Native execution
- Return value conversion
- Argument tree deallocation

Use blocks for bulk data. Blocks pass a direct pointer - no copying. This is the fastest path for large buffers. Integers, floats, and booleans are also low-overhead (simple struct fill).

Avoid per-element FFI calls. Don't call an FFI function once per array element. Instead, pass the whole array or block in a single call and do the loop in C.

```
// Bad — N FFI calls
foreach (x in bigArray) {
    let r = NativeProcess(x);
    ...
}

// Good — 1 FFI call
let result = NativeProcessAll(bigArray);
```

Prefer scalars over deep structures. Nested arrays of objects are recursively deep-copied. If you only need a few values, pass them as separate int/float arguments.

---

13. Callbacks - Calling Back into Flaris

A plugin can call a Flaris function at any point during its own execution. This is the foundation for event-driven patterns: a plugin fires `"ondata"` when a buffer is ready, `"onerror"` on failure, `"onprogress"` periodically - and the corresponding Flaris functions handle the result.

How it works

The VM maintains a flat registry mapping callback names to Flaris functions. When `Ffi.SetCallback` is called, the VM registers the function and passes a single dispatcher pointer - `flaris_dispatch` - to the plugin. The plugin stores that pointer and calls it by name whenever it wants to invoke Flaris code.

The call is **synchronous**: the plugin's C stack is suspended while the Flaris function executes, and the return value comes back as an `FfiObject`.

Registering a callback

```
fn onData(payload) {
    Console.WriteLine("got: ", payload);
    return 1;  // acknowledge
}

let lib = Ffi.Load("./plugin.so", PLUGIN_SHA);
Ffi.SetCallback(lib, "mylib_ondata", onData);
```

`Ffi.SetCallback(handle, name, fn)` - registers `fn` under `name` for the library identified by `handle`.

Naming convention: callback names are scoped per library handle - two plugins can both register `"ondata"` without colliding. The key is derived from both the name and the library handle. Prefixing is still good practice for readability but is not required for correctness.

Plugin side - what to implement

The plugin must export exactly one symbol for setup:

```
typedef FfiObject (*FlarisDispatch)(const char *name, FfiObject *args, int argc);
static FlarisDispatch g_dispatch = NULL;

// Flaris calls this once at SetCallback time
void flaris_set_callback(FlarisDispatch dispatch) {
    g_dispatch = dispatch;
}
```

After that, calling into Flaris from anywhere in the plugin is:

```
FfiObject args[2];
args[0].type   = FFIVAL_INT;  args[0].ival = 42;
args[1].type   = FFIVAL_BOOL; args[1].ival = 1;

FfiObject result = g_dispatch("ondata", args, 2);
// use result, then free if string/array
```

Complete example

plugin.c

```
#include "ffi_object.h"
#include <stdlib.h>
#include <string.h>

typedef FfiObject (*FlarisDispatch)(const char *name, FfiObject *args, int argc);
static FlarisDispatch g_dispatch = NULL;
```

```
void flaris_set_callback(FlarisDispatch dispatch) {
    g_dispatch = dispatch;
}

// Called from Flaris: triggers the callback and returns its result
FfiObject ffi_process(FfiObject *args, int argc) {
    if (!g_dispatch) return (FfiObject){ .type = FFIVAL_NIL };

    // Build args to send into Flaris
    FfiObject cb_args[1];
    cb_args[0].type          = FFIVAL_STRING;
    cb_args[0].string.data   = strdup("chunk_ready");
    cb_args[0].string.length = (uint32_t)strlen(cb_args[0].string.data);

    FfiObject result = g_dispatch("ondata", cb_args, 1);
    // cb_args[0].string.data was consumed by the VM — do not free
    return result;
}
```

script.fls

```
const PLUGIN_PATH = "./plugin.so";
const PLUGIN_SHA  = "...sha256...";

fn onData(event) {
    Console.WriteLine("event: ", event);
    return 99;
}

fn Main() {
    let lib     = Ffi.Load(PLUGIN_PATH, PLUGIN_SHA);
    Ffi.SetCallback(lib, "mylib_ondata", onData);

    let process = Ffi.GetFunction(lib, "ffi_process");
    let result  = process();
    Console.WriteLine("returned: ", result);  // prints: 99
}
```

Choosing argument types for callbacks

The argument types you pass to `g_dispatch` follow the same rules as regular FFI return values. Some types are significantly cheaper than others:

Type	Overhead	Recommended for
`FFIVAL_INT`, `FFIVAL_FLOAT`, `FFIVAL_BOOL`	None	Status codes, counts, scalars
`FFIVAL_BLOCK`	Zero-copy pointer	Binary buffers, audio, image data

Type	Overhead	Recommended for
`FFIVAL_STRING`	One `strdup`	Event names, short messages
`FFIVAL_ARRAY`	Deep malloc per element	Avoid in frequent callbacks
`FFIVAL_OBJECT`	Deep malloc + key copies	Avoid in frequent callbacks

For high-frequency callbacks (e.g. audio frames, sensor ticks), prefer `int` / `float` for status and `block` for data payloads. Reserve `string` for low-frequency events like errors or lifecycle notifications.

Ownership of callback args

String data you pass into `g_dispatch` via `strdup` is consumed and freed by the VM - do not free it yourself after the call. Block memory is never freed by the VM (it is a pointer into VM-managed memory or your own buffer). Ints, floats, and bools require no cleanup.

Lifecycle

Registrations are scoped to the library handle you pass to `Ffi.SetCallback`, so two libraries can use the same callback name without colliding. Registering a new function under the same name, for the same library, replaces the previous one. Use `Ffi.RemoveCallback(handle, name)` to unregister one explicitly; everything still registered is released when the VM shuts down.

Libraries themselves are never unloaded during the process. There is no `Ffi.Unload`: `dlclose` would leave every symbol already resolved through `Ffi.GetFunction` pointing into unmapped memory, and whether it unloads at all depends on what the library links, so reloading a rebuilt `.so` in-process is not supported. Restart the process, or isolate the plugin in a child process you can restart.

Async calls

A native call runs to completion on the single VM thread, so a slow one (a blocking query, a heavy transform) freezes every fiber for its duration. `Ffi.CallAsync` runs the call on the I/O thread pool instead: the calling fiber suspends and the scheduler keeps running other fibers until the result is ready.

```
let lib  = Ffi.Load("./codec.so", "<fingerprint>")
let enc  = Ffi.GetFunction(lib, "encode", "fn(array, int):array")

// Parks this fiber; other fibers run while `encode` executes off-thread.
let out  = await Ffi.CallAsync(enc, [frame, quality])

// Scheduled together, these run in parallel on the pool, not one after another.
let a = encodeJob(frame1)   // an `async fn` wrapping Ffi.CallAsync
let b = encodeJob(frame2)
Scheduler.Schedule(a); Scheduler.Schedule(b)
let ra = await a; let rb = await b
```

The offloaded function must be **pure**: it sees only its `FfiObject` arguments, must not call back into Flaris, must not touch shared mutable state, and must be thread-safe (several calls can run at once). Internally it may use as many threads as it likes, as long as it returns one `FfiObject`. This is the plugin author's contract; a function that calls back anyway is refused by the callback trampoline - the dispatch is dropped and a warning printed - rather than being allowed to touch VM state from the worker.

Two rules the runtime enforces: arguments marshal as **owned copies** (so the worker never reads VM memory), and **block arguments are refused** — blocks alias VM memory and cannot cross to a worker thread. A declared return type that does not match the plugin's result resolves to `nil`, the same failure signal the async `File.*` builtins use.

Off-thread is concurrency, not isolation: a crash in an offloaded call is still fatal to the process. Use this for functions that block, not to contain untrusted code.

ABI version

`FfiObject`'s layout is shared between the VM and every plugin, so a plugin built against a different revision of `ffi_object.h` would misread its arguments - undefined behaviour rather than a clean error. Every plugin must therefore invoke `FLARIS_PLUGIN_ABI()` once at file scope:

```
#include "ffi_object.h"

FLARIS_PLUGIN_ABI()          // exports flaris_abi_version()
```

`Ffi.Load` checks it. A plugin reporting a version **newer** than this runtime knows is refused with `Exception.ChecksumError` and a message naming both versions - it may set a struct member the runtime cannot see. An **older** version is accepted: revisions only append, so an old plugin's fields all still mean what they meant, and it needs no rebuild. A library exporting nothing is loaded with a warning, because it cannot be told apart from a shared library that was never a plugin - but every Flaris plugin is expected to carry the macro, and that leniency may be removed.

ABI	change
1	initial
2	`FFIVAL_POINTER` (opaque handles). Appended to the enum, and its union member fits inside the existing `table` member, so `sizeof(FfiObject)` is unchanged and v1 plugins keep working unmodified.

This is separate from `Ffi.GetVersion` / `flaris_version`, which reports *your plugin's* own version and is yours to define; the ABI version describes the marshalling contract and is owned by the runtime.

Guarantees and limits

- **One thread per VM.** All marshalling runs on the thread of the VM that made the call. A plugin must call `flaris_dispatch` (the callback trampoline) only from the thread that called into it - never from a thread it spawned. Nothing in the marshalling layer is locked, and it does not need to be: no two VMs share any of it.
- **Several VMs, one library.** A host may run one VM per thread, and each loads libraries for itself. If two of them load the same library, they share the mapping but not the call: the plugin's own code can then run on two threads at once, so anything it keeps in globals has to be thread-safe. Callback registrations are never shared - a name a VM registers resolves only for that VM, so the same name may mean a different function in each.
- **Handles.** An `Ffi.Load` handle stays valid for the lifetime of the VM that loaded it, and the library stays mapped for as long as any VM holds it. A handle belongs to that VM: passing it to another one, or passing anything that did not come from `Ffi.Load` at all - including a plugin's own `FFIVAL_POINTER` handle - raises instead of being used to resolve a symbol.

- **Handle lifetime is the plugin's.** The VM never frees what an `FFIVAL_POINTER` points at, and has no way to know when script drops the last reference. A plugin handing out handles must expose a close entry point; using a handle after close is a use-after-free the runtime cannot detect.
- **Exceptions.** A Flaris callback that raises unwinds the native frames back to the `Ffi` call that entered the plugin, and the exception continues to propagate in script. Native code between those two points does **not** get to clean up: destructors, `free` calls and locks in the abandoned C frames are skipped. Keep callback-invoking C code free of state that needs unwinding.
- **Crashes are fatal.** There is no signal handler. A plugin that segfaults, aborts or corrupts the heap takes the whole VM down; no exception is raised and no diagnostic is produced.
- **No raw C ABI.** A plugin function must have the shape `FfiObject f(FfiObject *args, int argc)`. You cannot point `Ffi.GetFunction` at an arbitrary C function such as `sqlite3_open` - it needs a shim compiled against `ffi_object.h` (see `ffi/sqlite_ffi.c`).

Limitations

- Callbacks are synchronous - the plugin blocks until the Flaris function returns.
- Callbacks cannot be called from a thread other than the one running the VM that registered them.
- The Flaris function must be a plain `fn` - not an async function or fiber.
- A maximum of 16 arguments can be passed per callback invocation.

---

14. Returning Functions from Flaris FFI

A FFI function can return another C function as a callable value back to Flaris. The returned value becomes a first-class `ffi-function` object - it can be stored, passed as arguments, returned from Flaris functions, or placed in arrays.

Basic Example

```
FfiObject ffi_printf(FfiObject *args, int argc) {
    (void)args;
    (void)argc;
    printf("Hello from ffi_printf!\n");
    return ffi_nil();
}

FfiObject ffi_func_return(FfiObject *args, int argc) {
    (void)args;
    (void)argc;

    FfiObject v;
    memset(&v, 0, sizeof(v));
    v.type = FFIVAL_FUNC;
    v.fn   = (FFIFunc)&ffi_printf;
    return v;
}
```

```
let ffi        = Ffi.Load("./funcs.so", "<fingerprint>");
let ffi_get_fn = Ffi.GetFunction(ffi, "ffi_func_return");
let retf       = ffi_get_fn();   // returns an ffi-function handle
retf();                          // calls ffi_printf - prints "Hello from ffi_printf!"
```

A returned function arrives as an **ffi** function, not a Flaris `fn`. If you annotate the factory, its return type is `callable` (or `ffi`) - `fn` matches only script functions and will raise `Exception.TypeMismatch`:

```
let ffi_get_fn = Ffi.GetFunction(ffi, "ffi_func_return", "fn():callable");
```

The returned function itself carries no signature, so it is always called fully dynamically: its arguments and return value are not type-checked.

Factory Pattern

The most useful application: the C side selects which function to return at runtime, giving Flaris dynamic dispatch without knowing symbol names upfront.

```
FfiObject fast_handler(FfiObject *args, int argc) { /* ... */ return ffi_nil(); }
FfiObject safe_handler(FfiObject *args, int argc) { /* ... */ return ffi_nil(); }

FfiObject ffi_get_handler(FfiObject *args, int argc) {
    FfiObject v;
    memset(&v, 0, sizeof(v));
    v.type = FFIVAL_FUNC;

    if (argc > 0 && args[0].type == FFIVAL_STRING &&
        strcmp(args[0].string.data, "fast") == 0) {
        v.fn = (FFIFunc)&fast_handler;
    } else {
        v.fn = (FFIFunc)&safe_handler;
    }
    return v;
}
```

```
let get_handler = Ffi.GetFunction(ffi, "ffi_get_handler");
let handler     = get_handler("fast");
handler();
```

First-Class Usage

Returned functions behave like any other callable:

```
let fns = [get_handler("fast"), get_handler("safe")];
fns[0]();
```

```
let pick = fn(mode) => get_handler(mode);
pick("safe")();
```

---

15. Best Practices & Safety

On the C side:

- Always validate `argc` and `args[i].type` before accessing values - never assume.
- Return a nil `FfiObject` on any validation failure rather than crashing.
- Use `memset(&out, 0, sizeof(out))` before setting fields on return values.
- Never store `FfiObject *` pointers past the function call - they are freed immediately after.
- Never free `block.memory` or any incoming `string.data`.
- Any string you return must be `malloc`'d - the VM calls `free()` on it.

On the Flaris side:

- Always check that `Ffi.Load` and `Ffi.GetFunction` returned non-nil before calling.
- Store the hash as a constant to avoid typos:

```
const PLUGIN_SHA = "a3f19c...64chars...";
let lib = Ffi.Load("./plugin.so", PLUGIN_SHA);
```

- Keep FFI calls coarse-grained - one call per batch, not one call per element.

Build discipline:

- Always use `ffi_object.h` from the same FlarisVM version you're running against. The struct layout must match exactly.
- Compile with `-fPIC` - required for shared libraries on all POSIX platforms.
- Treat plugin code with the same rigor as any native code - it runs in the same process as the VM with no sandboxing.

For the Ffi module API summary, see the Reference R8 section. For Buffer and Memory modules used alongside FFI, see R8 Buffer and Memory.